



UD3

INTRODUCCIÓN A JAVASCRIPT

3.4 MANEJO DE DATOS

MP_0373

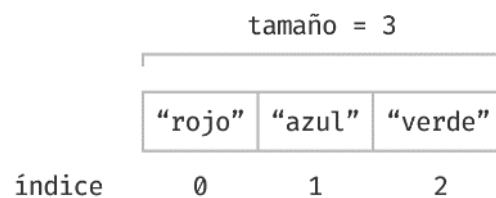
Lenguaje de marcas y
Sistemas de gestión de
la información

Introducción

Gracias a determinado tipo de datos que incluye JavaScript como los arrays y objetos se hace muy sencillo para el programador manejar gran cantidad de datos.

Arrays

Array es una colección de elementos que se guardan juntos con el fin de hacer operaciones como recorridos, mutaciones u ordenación.



La manera más sencilla de crear un array es utilizar los caracteres [] y asignar dicho array a una variable.

```
const arrayVacio = [];  
const colores = ["rojo", "verde", "azul"];  
  
// Otra manera de crear un array  
const arrayVacioOldSchool = new Array();  
const numerosOldSchool = new Array(1, 2, 3);
```

Una vez declarado un array, se puede acceder a sus elementos utilizando un índice. Para ello, basta con escribir el nombre del array y el índice entre dos []. Los arrays en JavaScript empiezan en el índice 0 y tendrán tantos índices como elementos - 1:

```
// Hay 3 colores en el array ocupando los índices 0, 1 y 2  
const colores = ["rojo", "azul", "verde"];  
console.log(colores[0]); // rojo  
console.log(colores[1]); // azul  
console.log(colores[2]); // verde  
console.log(colores[3]); // undefined
```

Una vez creado un array es posible añadir o quitar elementos con los métodos propios del array para tal caso:

- ⊕ `push()` Añade un elemento al final del array.
- ⊕ `unshift()` Añade un elemento al principio del array.
- ⊕ `pop()` Quita el último elemento de un array.
- ⊕ `shift()` Quita el primer elemento de un array.

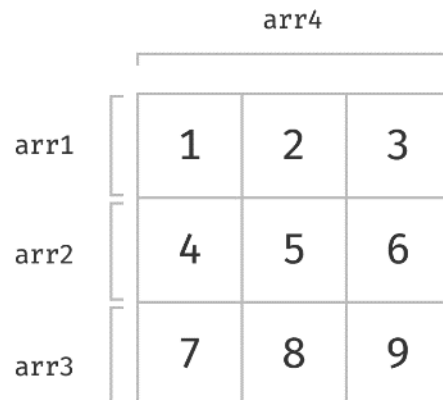
```
const array = [];  
array.push("rojo");  
array.push("azul"); // push Añade un elemento al final de array  
console.log(array); // rojo, azul  
array.unshift("verde");  
console.log(array) // verde, rojo, azul  
array.pop();  
console.log(array) // verde, rojo  
array.shift();  
console.log(array) // rojo
```

Además, es posible reemplazar un elemento que ya pertenezca al array sabiendo su índice, simplemente se debe realizar una asignación:

```
const colores = ["rojo", "azul", "verde"];  
colores[1] = "violeta";  
console.log(colores) // rojo, violeta, verde
```

Arrays multidimensionales

Un array multidimensional es un array de arrays, es decir, un array cuyos elementos son otros arrays. Los arrays multidimensionales se tratan de la misma manera que cualquier array, tanto el array principal como los arrays que lo componen, simplemente cambia el modelo mental del programador al utilizarlos.



```
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const arr3 = [7, 8, 9];
const arr4 = [arr1, arr2, arr3]; // Array multidimensional
```

Los arrays multidimensionales se recorren como cualquier otro array, simplemente se debe entender que cada elemento de este tipo de array es otro array y por lo tanto también se puede recorrer. No hay límite de dimensiones, simplemente es un límite de memoria.

```
const multiarray = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];

// Recorrido del primer nivel del array: iterador i
for (i = 0; i < multiarray.length; i++) {
  // Para cada nivel, se recorre el array de ese nivel: iterador j
  for (j = 0; j < multiarray[i].length; j++) {
    // Se utilizan los dos iteradores para acceder a cada elemento
    console.log(multiarray[i][j]);
  }
}

// Mostrará por consola
// 1 2 3 4 5 6 7 8 9
```

Métodos y propiedades de arrays

Length

Gracias a la propiedad `length` se puede consultar el tamaño de un array. Como los arrays empiezan en el índice 0, se puede decir que el último índice disponible de un array será su tamaño - 1.

```
let array = [];  
console.log(array.length); // 0  
array = [1, 2, 3, 4, 33];  
console.log(array.length); // 5  
console.log(array[array.length - 1]); // 33
```

Foreach

Gracias al método `forEach` incluido en los arrays de JavaScript, se puede iterar un array de una manera muy sencilla, al igual que se hace con un bucle `for` pero sin tener que mantener el índice.

`forEach` es un método que necesita como argumento una función. Dicha función recibe como parámetro cada uno de los elementos del array en cada una de las vueltas, es decir, esa función se ejecuta tantas veces como elementos tenga el array y en cada ejecución su primer parámetro es el elemento actual.

```
const colores = ["rojo", "verde", "azul"];  
colores.forEach(function(value) {  
    // En value está el elemento del array correspondiente  
    // En la primera ejecución de esta función value = rojo  
    // Segunda ejecución: value = verde  
    // Tercera ejecución: value = azul  
    console.log(value);  
});  
  
// Mostrará por consola  
// rojo verde azul
```

Lo suyo sería usar en estas funciones, [arrow Functions](#).
del PDF anterior

El método `forEach` recorrerá desde el primer hasta el último elemento del array ejecutando la función para cada elemento, de manera que el programador no debe preocuparse por el índice.

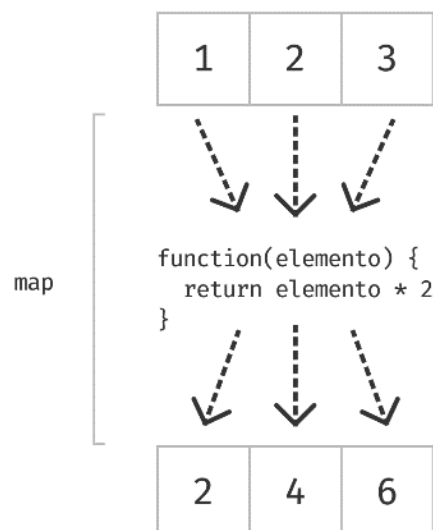
Ese índice se puede consultar ya que le llega a la función como el segundo parámetro (es opcional utilizarlo).

```
const colores = ["rojo", "verde", "azul"];
colores.forEach(function(value, i) {
  console.log(value);
  console.log(i);
});
```

```
// Mostrará por consola
// rojo 0 verde 1 azul 2
```

Map

El método map mapea un array: para cada elemento lo transforma con una función y devuelve el resultado en otro array.



```
const numeros = [1, 2, 3, 4, 33];

const numerosPor2 = numeros.map(function(elemento) {
  return elemento * 2;
})

console.log(numerosPor2); // 2, 4, 6, 8, 66
```

Find

El método find permite buscar elementos dentro de un array. Este método revisará cada uno de los elementos y parará de buscar una vez haya encontrado aquel elemento que está buscando, esto quiere decir que o bien no encuentra nada o bien encuentra un elemento, pero no encontrará más de un elemento. Como otros métodos de este tipo, no modifica el array, si no que devuelve el resultado.

Este método necesita una función como argumento que utilizará por cada uno de los elementos del array. Esa función tiene como parámetro cada uno de los elementos del array y se puede comprobar por cada elemento si cumple la condición de búsqueda: si la función devuelve true, dará por encontrado el elemento, sin embargo, si la función devuelve seguirá buscando.

```
const numeros = [1, 2, 3, 4, 33];

// Búsqueda con fin y función tradicional
const numeroMayorQueCuatro = numeros.find(function (elemento) {
  return elemento > 4;
});                                     Encuentra el primer elemento, que cumple el requisito

console.log(numeroMayorQueCuatro); // 33

// Utilizando arrow function
const numeroMayorQueCuatro = numeros.find(elemento => elemento > 4);
console.log(numeroMayorQueCuatro); // 33

// Dejando la función en una constante
const mayorQueCuatro = elemento => elemento > 4;
const numeroMayorQueCuatro = numeros.find(mayorQueCuatro);
console.log(numeroMayorQueCuatro); // 33
```

Filter

El método filter es similar a find. Su diferencia es que puede encontrar todos los elementos que cumplan una condición: esa condición es la que decida la función que se le pasa como argumento.

```
const numeros = [1, 2, 3, 4, 16, 33, 1502];
```

Nos devuelve todos los que cumplen esa condición

```
const pares = numeros.filter(function(numero) {
  return numero % 2 === 0;
});

console.log(pares); // 2, 4, 16, 1502
```

Sort

El método sort ordena un array siguiendo la condición que marque la función que se le pasa como argumento. La función de ordenación recibe dos parámetros que son utilizados para hacer precisamente la comparación como si fuesen dos elementos del array.

```
const numeros = [33, 45, 1, 100, 2, 4];

const ordenados = numeros.sort(function(a, b) {
  return a - b;
});

console.log(ordenados); // 1, 2, 4, 33, 45, 100
```

Una función de comparación function(a,b) devuelve:

- ⊕ < 0. Si el valor devuelto por la función de comparación es menor que 0, entonces el elemento b es mayor que a.
- ⊕ Si el valor devuelto por la función de comparación es 0, entonces a y b deben tener la misma posición.
- ⊕ 0. Si el valor devuelto por la función de comparación es mayor que 1, entonces el elemento.

Reduce

El método reduce como su nombre indica, reduce un array a una variable, es decir, de una forma similar al map, pasa por todos los elementos de un array y ejecuta una función.

La función recibe por parámetro el elemento actual del array y un acumulador, ese acumulador es lo que al final se reduce del array, es decir, para cada elemento del array transforma el acumulador que ya previamente han transformado los elementos anteriores y que transformarán los elementos siguientes. Para que el acumulador pase al siguiente elemento, la función debe devolver la transformación que le ha hecho al acumulador.

```
const numeros = [33, 45, 1, 100, 2, 4];
```

```
const suma = numeros.reduce(function(acumulador, numero) {  
  return acumulador + numero;  
});
```

```
console.log(suma); // 185
```

Operaciones con todos los
elementos de un array

Objetos

Los objetos son un tipo de dato para almacenar múltiples colecciones de datos mediante clave/valor.

```
const persona = {  
  nombre: 'Mercer',  
  apellido: 'Roussel',  
  direccion: 'Cruce Casa de Postas, 33'  
};
```

Cada una de las propiedades del objeto: claves, funciona como una variable declarada dentro del objeto, de manera que la clave es el nombre de la variable y el valor es el contenido de la variable que puede ser de cualquier tipo: un string, booleano, un array, incluso otro objeto.

Para acceder a las propiedades de un objeto se puede utilizar la misma sintaxis que para acceder a un array o mediante el carácter «:».

```
const persona = {  
  nombre: 'Mercer',  
  apellido: 'Roussel',  
  direccion: 'Cruce Casa de Postas, 33'  
};  
  
console.log(persona['nombre']); // Mercer  
console.log(persona.apellido); // Roussel  
  
// Si la propiedad no existe, se devuelve undefined  
console.log(persona['xyz']); // undefined
```

Métodos de objetos

En JavaScript una variable puede contener una función, por lo que una propiedad de un objeto puede contener una función:

```
const persona = {  
  nombre: 'Mercer',  
  apellido: 'Roussel',  
  saludar: function() { console.log('Soy Mercer') }  
}  
persona.saludar(); // Soy Mercer
```

De esta manera se añaden nuevas funcionalidades al objeto para hacerlo un poco más eficaz al utilizarse. Esas funciones predefinidas pueden incluso acceder al propio objeto en sí, utilizando para ello la palabra reservada `this`. `this` es una palabra reservada en JavaScript que se refiere al contexto de ejecución en el que se realiza una sentencia, en el caso de funciones dentro de objetos, `this` hace referencia al mismo objeto ya que es el contexto en el que se ejecuta dicha sentencia.

```
const persona = {  
  nombre: 'Mercer',  
  apellido: 'Roussel',  
  saludar: function() { console.log('Soy Mercer') }  
}
```

```
persona.saludar(); // Soy Mercer
```

Recorrer un objeto

Aunque un objeto está diseñado para albergar información de todo tipo, en determinados casos puede ser interesante recorrerlo (por ejemplo, para buscar información).

Es interesante destacar que JavaScript no asegura el orden de las claves de un objeto, porque un objeto en realidad está pensado para albergar información y recuperarla con una clave y no para almacenar información de manera ordenada (para eso están los arrays), por lo tanto, el orden en el que aparecen las claves en una iteración puede no ser el mismo orden en el que se definieron en el objeto.

```
// Bucle for ... in
const numeros = { a: 1, b: 2, c: 3 };

for (const clave in numeros) {
  console.log(`${clave}: ${numeros[clave]}`);
}

// Muestra por consola:
// a: 1
// b: 2
// c: 3

// Object.keys
// Devuelve un array con las claves del objeto de manera que se puede iterar
const numeros = { a: 1, b: 2, c: 3 };
Object.keys(numeros).forEach(function(clave){
  console.log(`${clave}: ${numeros[clave]}`);
});

// Muestra por consola:
// a: 1
// b: 2
// c: 3
```

Recorriendo de un array de datos consumidos desde API

Un programador JavaScript es muy habitual que necesite hacer llamadas a una API. Una API es un servicio que expone un servidor mediante al cual se pueden realizar peticiones con el fin de recuperar o almacenar datos, realizar acciones, etc.

Una llamada a API muy habitual se realiza para recoger datos, por ejemplo, un listado de usuarios. Habitualmente esos listados se entregan como un array de objetos: cada objeto es una entidad y el array los contiene.

```
const listadoUsuarios = [  
  {  
    nombre: "Gabriel Pacheco",  
    edad: 34,  
    signoZodiaco: "Virgo"  
  },  
  {  
    nombre: "Lola Rocha Tercero",  
    edad: 30,  
    signoZodiaco: "Capricornio"  
  },  
  ...  
];
```

Gracias a los métodos de los que disponen los arrays en JavaScript, se pueden realizar búsquedas o transformaciones habituales de dichos listados:

```
// Obtener solo nombres  
const soloNombres = [];  
listadoUsuarios.forEach(function(usuario){  
  soloNombres.push(usuario.nombre);  
});  
  
// Obtener solo nombres (manera más sencilla)  
const soloNombres2 = listadoUsuarios.map(function(usuario) {  
  return usuario.nombre;  
});
```

```
// Edad total (suma de todas las edades)
// Con arrow function
const edadTotal = listadoUsuarios.reduce((acc, usuario) => acc +
usuario.edad, 0);

// El segundo parámetro que recibe reduce es el valor inicial del acumulador
en este caso, es 0

// Obtener solo los usuarios con signo Virgo
const soloVirgo = listadoUsuarios.filter(function(usuario) {
  return usuario.signoZodiaco === 'Virgo';
});
```